The Eholoko Simulation Engine: Architecture, Discretization, and Analysis Protocols for a Unified Field

Tshuutheni Emvula*

January 2, 2026

Abstract

The Eholoko Fluxon Model (EFM) represents a paradigm shift from analytical, equation-fitting physics to computational, emergent physics. Unlike the Standard Model, which inputs particle masses and constants as free parameters, the EFM derives these properties as emergent outputs of a single, deterministic scalar field simulation. This paper details the foundational methodologies of the EFM Simulation Engine. We define the numerical discretization of the density-dependent Nonlinear Klein-Gordon (NLKG) equation, the GPU-accelerated state-switching architecture used to model multi-regime physics, and the rigorous "Rosetta Stone" protocol for scaling dimensionless simulation units to physical observables. By establishing these protocols, we provide the necessary framework for reproducing the model's claims regarding cosmogenesis, particle mass spectra, and galactic dynamics.

*Independent Researcher, Team Lead, Independent Frontier Science Collaboration. Validated against the EFM Compendium v1.3.

Contents

1 Introduction: The Grid as Ontology

In standard theoretical physics, the "simulation" is often a tool used to approximate a solution to an analytical equation that cannot be solved by hand (e.g., Lattice QCD). In the Eholoko Fluxon Model (EFM), the simulation *is* the theory. The EFM posits that physical reality is not continuous but is fundamentally discrete, emergent from the harmonic interactions of a scalar field, ϕ, on a quantized grid.

Therefore, the construction of the simulation engine is not merely a coding exercise but an ontological definition of the universe. The engine is built upon three non-negotiable axioms:

1. **Deterministic Evolution:** Given an initial state $\phi(t_0)$, the state $\phi(t)$ is fully determined by the non-linear dynamics of the field itself, without stochastic injection.
2. **State-Dependent Physics:** The coefficients of the governing equation are not global constants. They are local functions of the field density, $\rho = k\phi^2$. This allows the vacuum, matter, and quantum regimes to coexist and interact within the same grid.
3. **Dimensionless Operation:** The engine operates in pure number space ("sim-units"). Physical units (meters, seconds, kilograms) are only applied *post-hoc* via a rigorous scaling protocol, preventing the circular reasoning of fine-tuning inputs to match outputs.

This paper documents the methods used to achieve the results presented in the EFM Compendium, focusing on the high-performance JAX and PyTorch implementations used in the 'Cosmogenesis' and 'Structure' simulation series.

2 The Governing Dynamics: Discretization of the NLKG

The core of every EFM simulation is the numerical integration of the generalized, density-dependent Nonlinear Klein-Gordon (NLKG) equation:

$$\frac{\partial^2\phi}{\partial t^2} - c^2\nabla^2\phi + m^2(\rho)\phi + g(\rho)\phi^3 + \eta(\rho)\phi^5 - \alpha\phi\frac{\partial\phi}{\partial t}\cdot\nabla\phi - \delta\left(\frac{\partial\phi}{\partial t}\right)^2\phi = 0 \tag{1}$$

This equation is not solved analytically. It is discretized spatially and temporally to evolve the state tensor $\Phi^n_{i,j,k}$.

2.1 Spatial Discretization via Convolution

The continuous Laplacian $\nabla^2\phi$ is discretized onto a 3D cubic grid using a finite-difference stencil. For high-performance GPU execution (typically on NVIDIA A100s), we utilize 3D convolution operations. The standard 7-point stencil is defined as:

$$\nabla^2\phi \approx \frac{1}{dx^2}\sum_{i,j,k} w_{i,j,k}\phi_{x+i,y+j,z+k} \tag{2}$$

where the weights w represent the topological connectivity of the grid points. Periodic boundary conditions are enforced via padding (mode='wrap'), simulating an infinite, unbounded universe.

Methodological Note on Resolution: The grid resolution N is treated as a physical parameter, not just numerical precision. Convergence studies (e.g., the V28 Galaxy Convergence test) demonstrate that emergent properties like star formation efficiency scale with resolution. Definitive simulations require $N \geq 512^3$ to capture the harmonic hierarchy.

2.2 The Density-Dependent Switching Logic

The defining innovation of the EFM engine is the "State-Switching" logic. The simulation does not use global constants for mass (m) or interaction (g). At every timestep t, the engine computes the local density tensor:

$$\boldsymbol{\rho} = k \cdot (\boldsymbol{\phi} \circ \boldsymbol{\phi}) \tag{3}$$

Using this density tensor, boolean masks are generated to classify every voxel into a Harmonic Density State (HDS):

- **Vacuum Mask (M_{vac}):** $\boldsymbol{\rho} < \rho_{thresh}$. Applies repulsive $g > 0$ and low m to maintain cosmic expansion.
- **Matter Mask (M_{mat}):** $\boldsymbol{\rho} \geq \rho_{thresh}$. Applies attractive $g < 0$ and high m to drive soliton formation (particle creation).

The potential force term $F(\phi)$ is then computed as a masked superposition:

$$\mathbf{F}_{potential} = M_{vac} \circ F_{S/T}(\boldsymbol{\phi}) + M_{mat} \circ F_{S=T}(\boldsymbol{\phi}) \tag{4}$$

This logic allows a single simulation code to simultaneously model the expansion of the cosmic void and the condensation of galaxies within it, solving the "scale problem" of standard cosmology.

3 The "Rosetta Stone" Scaling Protocol

EFM simulations generate dimensionless data. Connecting these to physical reality requires the **Universal Scaling Laws**. We adhere to a strict **Anchoring Protocol** to ensure falsifiability.

3.1 The Principle of Anchoring

We cannot assume the values of constants. We must *derive* them by anchoring *one* emergent simulation feature to *one* known physical constant.

3.1.1 Cosmological Scaling (N1 State)

To derive the speed of light and cosmic time from the 'StructureV9' simulation:

1. **Simulate:** Run the universe until Large Scale Structure (LSS) emerges.
2. **Measure:** Identify the peak of the two-point correlation function: $r_{sim} = 1.99$.
3. **Anchor:** Equate this to the observed LSS scale: $L_{phys} = 628$ Mpc.
4. **Derive Length Scale (S_L):** $S_L = 628$ Mpc$/1.99 \approx 315.6$ Mpc/sim-unit.
5. **Derive Time Scale (S_T):** Using the simulation speed $c_{sim} = 1$, we enforce $S_T = S_L/c_{phys}$.

This derivation was validated by predicting the fundamental cosmic frequency (1.20×10^{-20} s^{-1}) and recovering the speed of light with 98.35% internal consistency.

3.1.2 Particle Scaling (N3 State)

To derive the energy density of the vacuum from the 'MassGeneration' simulation:

1. **Simulate:** Evolve a single soliton until stable.
2. **Measure:** Integrated mass proxy $M_{sim} = \int \phi^2 dV$.
3. **Anchor:** Equate to the physical electron mass $m_e \approx 9.11 \times 10^{-31}$ kg.
4. **Prediction:** This scaling allowed the prediction of the "Generative Vacuum" energy density ($\sim 10^{19}$ J/m^3), resolving the Cosmological Constant Problem by distinguishing it from the "Cosmic Vacuum" density.

4 Analysis Pipelines: Extracting Observables

Raw field data (Φ) is opaque. The EFM employs specific post-processing pipelines to translate tensor data into physical observables comparable with experimental datasets (e.g., Planck, DESI).

4.1 The Particle Census Pipeline

To extract the "Particle Zoo" from a continuous field, we utilize topological analysis:

1. **Thresholding:** A density cut ρ_{cut} isolates high-density regions from the background vacuum.
2. **Connected Components:** We employ GPU-accelerated labeling (e.g., `cupyx.scipy.ndimage.label`) to identify distinct topological islands.
3. **Integration:** For each label i, we integrate the field properties to determine Mass ($M \propto \phi^2$) and Charge Asymmetry ($Q \propto \phi^3$).

This method generates the mass histograms that revealed the quantized hadron spectrum in the 'Nucleosynthesis' simulations.

4.2 Spectral Analysis

For cosmological validation, we perform Fourier analysis on the density contrast field $\delta = (\rho - \bar{\rho})/\bar{\rho}$.

- **Spatial Power Spectrum $P(k)$:** Computed via 3D FFT of the simulation volume. The resulting $P(k)$ is spherically averaged and compared directly to the Planck CMB power spectrum.
- **Dynamic Spectra:** For time-varying signals (e.g., Pulsar Wind Nebulae), we calculate the spectrogram of the electric field analogue $\mathbf{E} = -\nabla A_0 - \partial \mathbf{A}/\partial t$ at the soliton core.

5 Conclusion

The Eholoko Simulation Engine provides a rigorous, self-consistent framework for simulating unified field physics. By combining a deterministic NLKG solver with density-dependent state switching and a strict anchoring protocol, the EFM avoids the parameter-fitting pitfalls of standard models. The code structure ensures that phenomena—from the mass of the neutron to the rotation of galaxies—are emergent properties of the grid dynamics, establishing the simulation itself as the proof of the theory.

A Conceptual Simulation Code (‘unified_solver.py‘)

The following conceptual code illustrates the core JAX-based logic for the unified, density-dependent solver used in the ‘Cosmogenesis‘ and ‘Structure‘ simulations.

```python
import jax
import jax.numpy as jnp
from jax.scipy.signal import convolve

@jax.jit
def unified_solver_step(phi, phi_dot, dt, dx, params):
    """
    Performs one timestep of the EFM universe evolution using
    density-dependent masking.
    """
    # 1. Calculate Local Density Tensor
    rho = params['k'] * (phi ** 2)

    # 2. State-Switching Logic (The Masks)
    # Boolean tensors separate Vacuum (S/T) from Matter (S=T)
    is_matter = rho > params['rho_critical']
    is_vacuum = ~is_matter

    # 3. Apply State-Dependent Parameters dynamically
    # Mass term: Vacuum is stable/light, Matter is heavy/confining
    m_sq = (params['m_vac'] * is_vacuum) + (params['m_mat'] * is_matter)

    # Nonlinearity: Vacuum is repulsive (+g), Matter is attractive (-g)
    g = (params['g_vac'] * is_vacuum) + (params['g_mat'] * is_matter)

    # 4. Compute Spatial Laplacian (3D Tensor Convolution)
    # Using a 7-point stencil for finite difference
    stencil = jnp.array([[[0,0,0],[0,1,0],[0,0,0]],
                         [[0,1,0],[1,-6,1],[0,1,0]],
                         [[0,0,0],[0,1,0],[0,0,0]]]) / dx**2
    laplacian = convolve(phi, stencil, mode='wrap', precision=jax.lax.Precision
        .HIGH)

    # 5. Compute Forces (NLKG terms)
    # F = m^2*phi + g*phi^3 + eta*phi^5 ...
    force = (m_sq * phi) + (g * (phi ** 3)) + (params['eta'] * (phi ** 5))

    # 6. Evolve Field (Verlet Integration for Energy Stability)
    acceleration = laplacian - force
    phi_new = (2 * phi) - phi_prev + (acceleration * (dt ** 2))

    return phi_new, phi
```

Listing 1: The Unified Density-Dependent EFM Solver Loop

References

[1] T. Emvula, *Compendium of the Eholoko Fluxon Model*. Independent Frontier Science Collaboration, 2025.

[2] T. Emvula, ”Dimensionless Parameters and Universal Scaling in the Ehokolo Fluxon Model,” Independent Frontier Science Collaboration, 2025.

[3] T. Emvula, ”Cosmogenesis in the Ehokolo Fluxon Model: Emergent Particles and a

Solution to the Cosmological Constant Problem," Independent Frontier Science Collaboration, 2025.

[4] T. Emvula, "From Nebula to Galaxy: A First-Principles Derivation of Structure Formation," Independent Frontier Science Collaboration, 2025.

[5] T. Emvula, "Ehokolon Harmonic Density States: Foundational Validation," Independent Frontier Science Collaboration, 2025.